

Übung 20.04. - 24.04.2026
Blatt 01

Besprechung der Aufgaben in der Woche vom 27.04.2026.



Aufgabe 1 [Comparable]

Schreiben Sie die Klasse **Password**.

- Sie besitzt das **String**-Attribut **value**, welches ein Passwort speichert.
- Sie besitzt den Konstruktor **Password(String value)**, welcher das Attribut setzt.
- Sie implementiert das Interface **Comparable<String>** (<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Comparable.html>).

Implementieren Sie die Methode **compareTo**. Diese vergleicht das gespeicherte Passwort **value** mit dem übergebenen Passwort hinsichtlich ihrer Sicherheit. Dabei gilt ein Passwort als „größer“, wenn es sicherer ist.

Der Vergleich erfolgt in zwei Schritten:

- Zuerst wird die Länge des Passworts verglichen. Ein längeres Passwort gilt als sicherer.
- Bei Gleichstand wird die Anzahl der Ziffern ('0'–'9') verglichen. Ein Passwort mit mehr Ziffern gilt als sicherer.

Bei weiterem Gleichstand gelten beide Passwörter als gleich sicher.

Beispiel:

- "abc" vs. "abcd" → "abcd" ist sicherer (größer), da es länger ist.
- "abc1" vs. "abcd" → gleich lang, aber "abc1" ist sicherer (größer), da es mehr Ziffern enthält.
- "ab12" vs. "cd34" → gleich lang und gleich viele Ziffern → beide sind gleich sicher.

Erzeugen Sie in der **main**-Methode mehrere **Password**-Objekte sowie verschiedene Vergleichswerte und testen Sie Ihre Implementierung durch Aufrufe von **compareTo**.

Hinweis:

Überlegen Sie sich, wie Sie mithilfe der Methoden **isDigit** ([https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Character.html#isDigit\(char\)](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Character.html#isDigit(char))) und **charAt** ([https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/String.html#charAt\(int\)](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/String.html#charAt(int))) bestimmen können, welches Passwort mehr Ziffern beinhaltet.

Aufgabe 2 [TimeUnit & TimeSpan]

Implementieren Sie das Beispielprogramm Zeiteinheit aus Vorlesung 1 (ab Folie 100).

- (a) Ergänzen Sie das Programm um eine neue Methode im Stil der **atLeast4000Seconds** Methode. Dieses Mal soll allerdings die Anzahl der Sekunden, die die **TimeUnit** mindestens haben soll, damit **true** zurückgegeben werden kann, als Übergabeparameter übergeben werden. Beispielsweise könnte man mit dem entsprechenden Übergabeparameter überprüfen, ob eine Zeiteinheit mindestens 300 statt 4000 Sekunden hat, und so weiter.

- (b) Überladen Sie die Methode aus (a), indem Sie einen weiteren Übergabeparameter in Form eines **char** ergänzen (also **atLeast(int amount, char c, TimeUnit unit)**). Dieser soll die Zeiteinheit festlegen, in der **amount** vorliegen wird. Dabei nimmt **c** entweder den Wert 's' für Sekunden, 'm' für Minuten oder 'h' für Stunden an.
- (c) Erweitern Sie das Programm um eine weitere Klasse **TimeSpan**, welche ebenfalls das Interface **TimeUnit** implementiert. **TimeSpan** besitzt die Attribute **Hours hours**, **Minutes minutes** und **Seconds seconds**, sowie einen Konstruktor **TimeSpan(int hours, int minutes, int seconds)**, der die Attribute entsprechend setzt.
- Beispiel:**
Ein Objekt **ts**, erzeugt mit **new TimeSpan(5, 10, 20)** entspricht 5 Stunden, 10 Minuten und 20 Sekunden und somit 18620 Sekunden. Ein Aufruf von **atLeast(256, 'm', ts)** liefert daher **true**.
- (d) Überlegen Sie sich sinnvolle Testfälle, mit denen Sie ihr Programm auf Korrektheit überprüfen können.

Aufgabe 3 [Shape]

Schreiben Sie das Interface **Shape**.

- Es besitzt die Methode **double getArea()**, welche den Flächeninhalt der Form zurückgibt.
- Es besitzt die Konstante **double PI = 3.141**, welche für Kreisberechnungen verwendet werden soll.

Schreiben Sie die Klassen **Circle** und **Rectangle**.

- **Circle** besitzt das Attribut **double radius** sowie einen passenden Konstruktor, der **radius** setzt.
- **Rectangle** besitzt die Attribute **double width** und **double height** sowie einen passenden Konstruktor, der **width** und **height** setzt.
- Beide Klassen implementieren das Interface **Shape**.

Schreiben Sie nun die Methode **createShape** in der Klasse **Main**.

- Modifikatoren: **public static**
- Rückgabetyt: **Shape**
- Name: **createShape**
- Übergabeparameter: **char type, double a, double b**

Diese Methode erzeugt abhängig vom Parameter **type** ein Objekt und gibt es zurück:

- Falls **type == 'c'**: Erzeuge ein **Circle**-Objekt mit Radius **a**.
- Falls **type == 'r'**: Erzeuge ein **Rectangle**-Objekt mit Breite **a** und Höhe **b**.

Falls ein unbekannter Typ übergeben wird, geben Sie **null** zurück.

Verwenden Sie die Methode in der **main**-Methode:

- Erzeugen Sie mehrere Formen mithilfe von **createShape**.
- Geben Sie jeweils die berechnete Fläche auf der Konsole aus.

Beispiele:

- Ein Aufruf **createShape('c', 2.0, 0)** erzeugt einen Kreis mit Radius 2.
- Ein Aufruf **createShape('r', 3.0, 4.0)** erzeugt ein Rechteck mit Fläche 12.

Hinweis: Achten Sie darauf, dass der Rückgabetyp der Methode **createShape** das Interface ist, nicht die konkreten Klassen.

Aufgabe 4 [Discountable]

Schreiben Sie das Interface **Discountable**.

- Es besitzt die Konstante **double DEFAULT_DISCOUNT = 0.1** (entspricht 10% Rabatt).
- Es besitzt die abstrakte Methode **double getPrice()**, welche den Originalpreis eines Objekts zurückgibt.
- Es besitzt die Default-Methode **double getDiscountPrice()**, welche den Preis nach Anwendung des Standardrabatts zurückgibt.

Die Default-Methode **getDiscountPrice** soll den Preis mithilfe der Konstanten **DEFAULT_DISCOUNT** berechnen.

Schreiben Sie die Klassen **Book** und **Electronic**.

- **Book** besitzt das Attribut **double price** sowie einen passenden Konstruktor, der **price** setzt.
- **Electronic** besitzt ebenfalls das Attribut **double price** sowie einen passenden Konstruktor, der **price** setzt.
- Beide Klassen implementieren das Interface **Discountable**.

Überschreiben Sie zusätzlich in der Klasse **Electronic** die Default-Methode **getDiscountPrice**, sodass für elektronische Geräte ein Rabatt von 20% gewährt wird.

Erzeugen Sie in der **main**-Methode mehrere Objekte der Klassen **Book** und **Electronic**.

- Geben Sie jeweils den Originalpreis sowie den rabattierten Preis aus.

Beispiel: Ein Buch mit Preis 50.0 hat einen rabattierten Preis von 45.0. Ein elektronisches Gerät mit Preis 100.0 hat einen rabattierten Preis von 80.0.

Hinweis: Greifen Sie in der Default-Methode direkt auf die Konstante des Interfaces zu und verwenden Sie die Methode **getPrice** zur Berechnung.